Mardi 10/11 — Grid, puis le mobile first

Kevin Bardoux

mardi 10 novembre 2026

Table des matières

Objectifs 2
Matin — Grid : démonstration et pas à pas ensemble 2
Rappel (5 min) 2
Apport — Grid (20 min) 3
Grid — attention et vérification 6
Pas à pas ensemble — La grille de projets (1 h) 6
Enregistrer et envoyer (2 min) 8
Exercices 1 et 2 — La galerie et la page (30 min) 8
Enregistrer et envoyer (2 min) 9
À retenir — ce matin 9
Si le temps le permet 9
Si le temps le permet — Grid Garden 10
Débrief 10
Après-midi — Le responsive : mobile first 10
Apport — Mobile first (15 min) 10
Mobile first — attention et vérification 14
Pas à pas ensemble — La page en deux largeurs (55 min) 14
Enregistrer et envoyer (2 min) 16
Exercices — La galerie et le menu (35 min) 17
« Layout Lab » : votre page personnelle (25 min) 17
Enregistrer et envoyer (2 min) 18
À retenir — cet après-midi 18
Si le temps le permet 18
Sources 19

Note

Compétences visées : C3 **Livrable** : `node --test grid-exercices.test.js` et `node --test responsive-exercices.test.js` au vert ; le Layout Lab commencé.

! Le minimum du jour

- Les deux pas à pas, faits avec le groupe.
- `node --test grid-exercices.test.js` : exercices 1 et 2.
- `node --test responsive-exercices.test.js` au vert.

Le reste (rubrique « Si le temps le permet » et encadrés « Défi ») est la suite naturelle, pour ceux qui avancent.

Objectifs

- Construire une grille à deux dimensions, et choisir entre Grid et Flexbox.
- Écrire des styles de base pour le téléphone, puis une media query en `min-width`.
- Vérifier une page à 375 et à 1280 px.

Comment se passe la journée. Une notion le matin (Grid), une notion l'après-midi (le *mobile first*). Chacune : une démonstration lente, un pas à pas ensemble, puis des exercices.

Matin — Grid : démonstration et pas à pas ensemble

Rappel (5 min)

1. Sur quel élément écrit-on `display: flex` ?
2. Quelle unité pour une taille de texte accessible ?
3. Que fait un `.gitignore` ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport — Grid (20 min)

- **Pourquoi** : ranger des cartes en lignes **et** en colonnes.
- **Grid** : un parent `display: grid` place ses enfants dans une grille.

Pourquoi. Une galerie de projets, une page avec en-tête, contenu et pied : il faut aligner dans les deux sens à la fois.

Définition. Grid organise en **deux** dimensions, lignes et colonnes (MDN Web Docs, s. d.-a, s. d.-b). `grid-template-columns` définit les colonnes ; `1fr` est une fraction de l'espace disponible. **Flexbox ou Grid ?** Flexbox pour une rangée d'éléments (un menu, une liste de badges) ; Grid pour une grille à deux dimensions (des cartes, la page entière). Les deux se combinent : une grille de cartes, un menu en Flexbox.

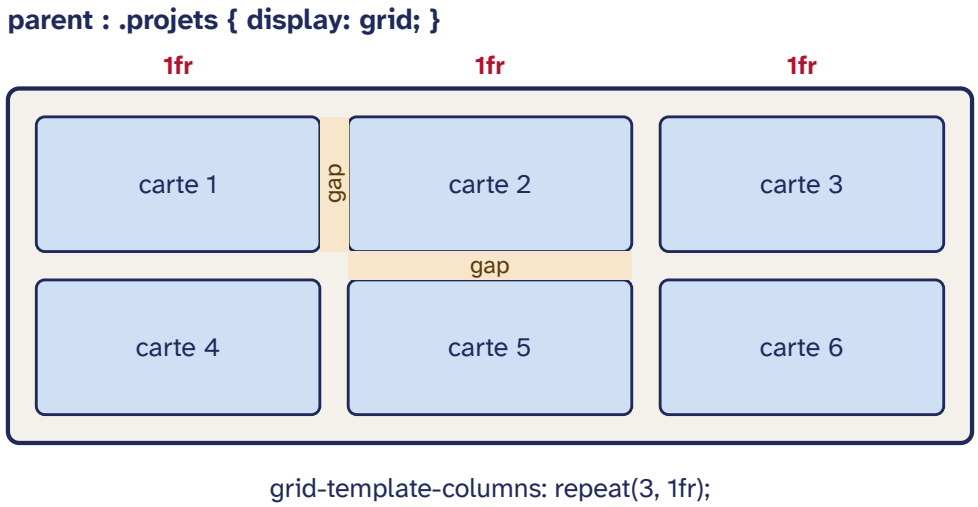


Figure 1. – Grid : `repeat(3, 1fr)` fait trois colonnes égales ; les cartes remplissent la grille ligne par ligne, avec le même `gap` dans les deux sens.

Démonstration en direct, à ouvrir dans le navigateur : grid-template-columns sur six cartes. Elle se règle au clavier (Tab, puis les flèches) ou à la souris.

Exemple.

```
.projets {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
```

```
  gap: 1rem;
}
```

Pas à pas.

1. `display: grid` sur le conteneur des projets.
2. `grid-template-columns: repeat(3, 1fr)` : trois colonnes égales.
3. `gap: 1rem` : l'espace entre les cartes, dans les deux sens.
4. F12 : cliquez sur le badge `grid` à côté de l'élément : la grille se dessine sur la page.

Quelques motifs courants de `grid-template-columns`, sur les mêmes six cartes :

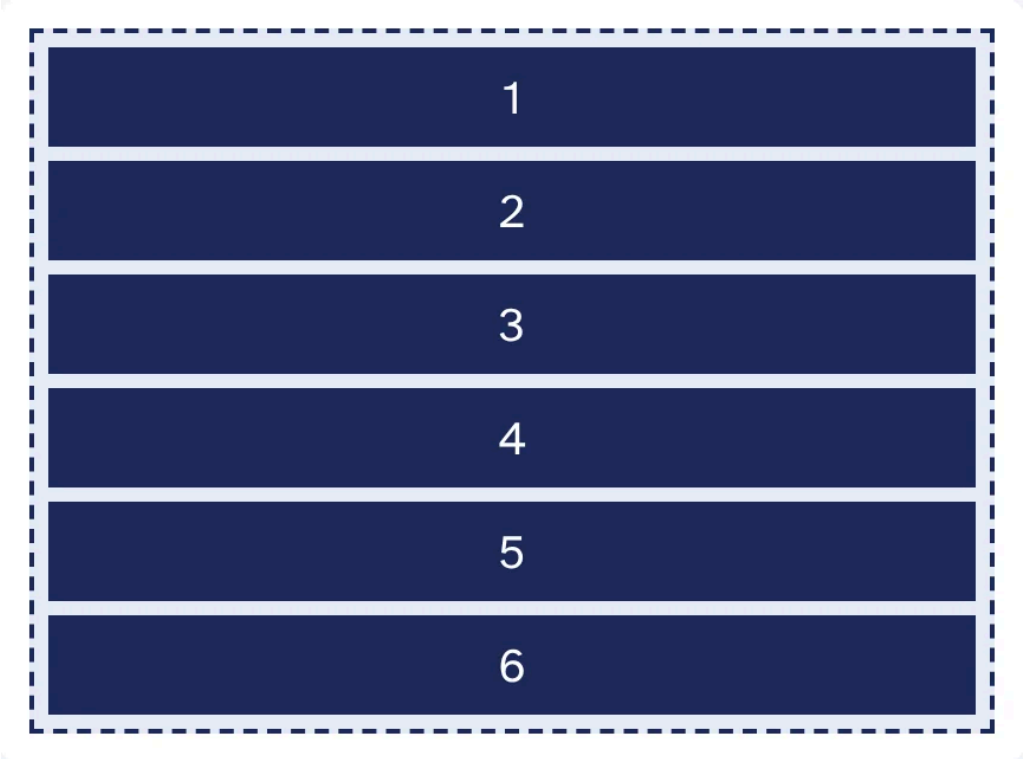


Figure 2. – `1fr`

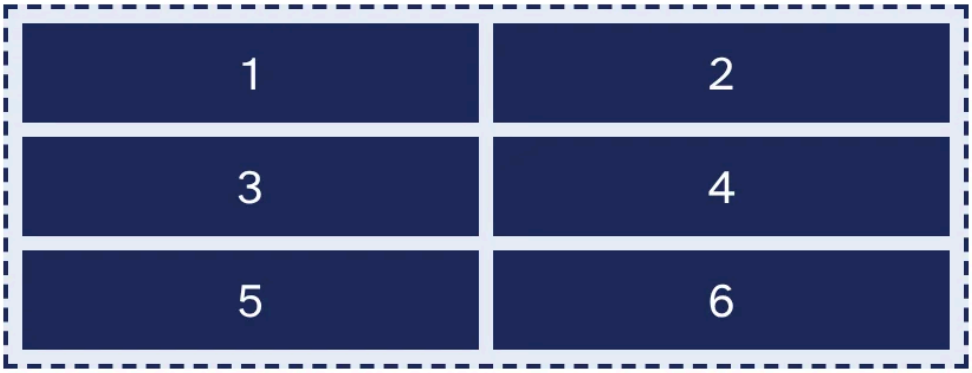


Figure 3. – `repeat(2, 1fr)`

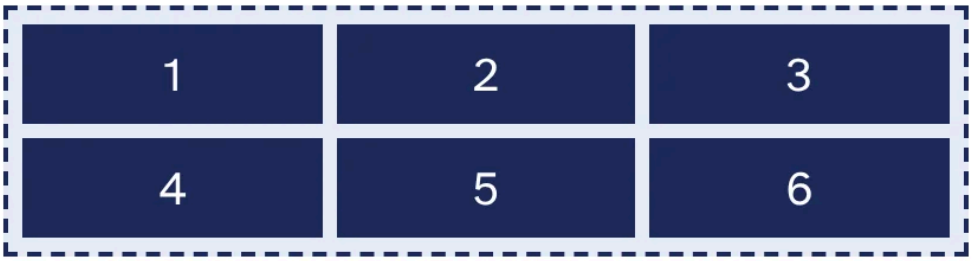


Figure 4. – `repeat(3, 1fr)`

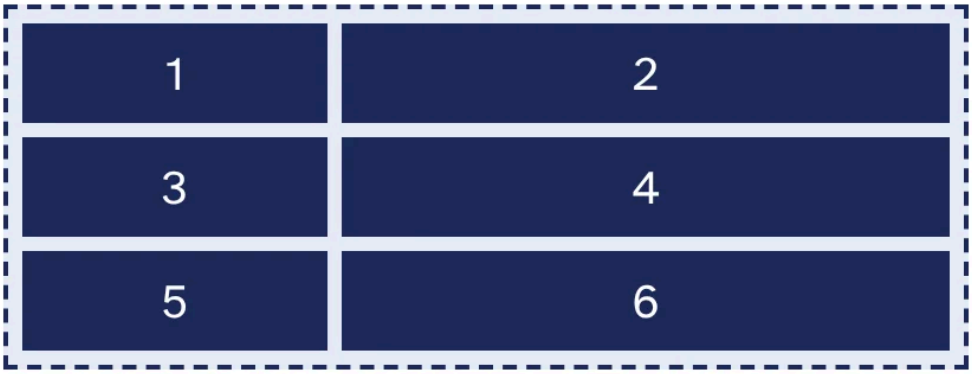


Figure 5. – `1fr 2fr`

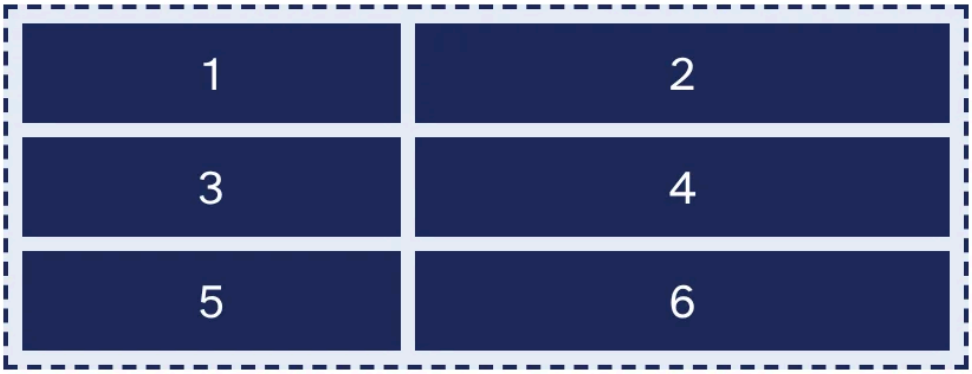


Figure 6. – `10rem 1fr`

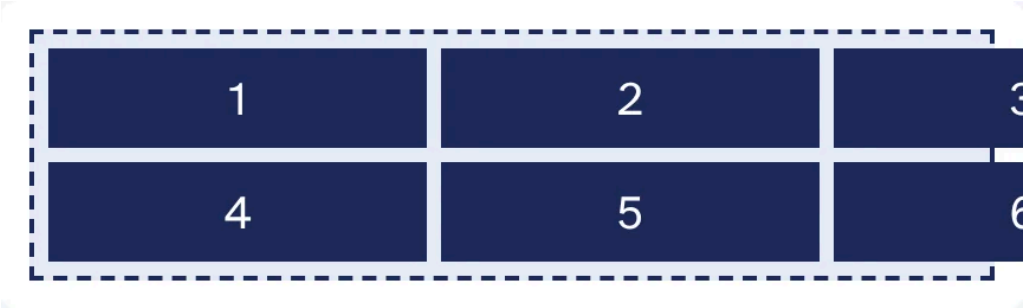


Figure 7. – `repeat(3, 10rem)`

Grid — attention et vérification

> ⚠ Attention
>
> Trois colonnes fixes sur un téléphone : les cartes deviennent minuscules. Une seule colonne au départ, plusieurs sur grand écran : c'est l'apport de cet après-midi.

- **Vérifiez** : Flexbox ou Grid pour une galerie de 9 photos ?

Pas à pas ensemble — La grille de projets (1 h)

- Je code, vous tapez en même temps : une étape à la fois.
- À chaque étape : une question, puis « Vous devez voir ».
- Perdu ? Dossier `rattrapage/grid/`, une étape à la fois.
- Dossier : `pas-a-pas-grid`.

Niveau : guidé. Ouvrez `pas-a-pas-grid/index.html` avec Live Preview, et `style.css` à côté. Tout se passe dans `style.css`, à la fin du fichier.

Étape 1. Passez `.projets` en Grid.

Avant d'enregistrer : l'écran va-t-il changer ?

```
/* Grid : la section .projets est le parent */
.projets {
  /* Grid : les enfants se placent en grille */
  display: grid;
}
```

Vous devez voir : rien ne change : par défaut, une seule colonne. Dans F12, le badge `grid` est apparu.

Perdu ? Copiez `rattrapage/grid/etape-1/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Étape 2. Définissez trois colonnes égales.

Avant d'enregistrer : combien de cartes par ligne ?

```
/* Grid : la section .projets est le parent */
.projets {
  /* Grid : les enfants se placent en grille */
  display: grid;
  /* Grid : trois colonnes égales, 1fr chacune */
  grid-template-columns: repeat(3, 1fr);
}
```

Vous devez voir : trois cartes par ligne, sur deux lignes, collées les unes aux autres.

Perdu ? Copiez `rattrapage/grid/etape-2/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Étape 3. Ajoutez un espace.

Avant d'enregistrer : dans quels sens l'espace va-t-il s'appliquer ?

```
/* Grid : la section .projets est le parent */
.projets {
  /* Grid : les enfants se placent en grille */
  display: grid;
  /* Grid : trois colonnes égales, 1fr chacune */
  grid-template-columns: repeat(3, 1fr);
  /* Grid : un espace entre les cartes */
  gap: var(--espace-3);
}
```

Vous devez voir : le même espace entre les colonnes et entre les lignes. À 375 px, les trois colonnes sont trop étroites et la page déborde : c'est ce que corrige le *mobile first*, cet après-midi.

Perdu ? Copiez `rattrapage/grid/etape-3/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Étape 4. Dans F12, cliquez sur le badge `grid`. Essayez `1fr 2fr` à la place de `repeat(3, 1fr)`, regardez, puis remettez `repeat(3, 1fr)`.

Avant d'enregistrer : que dessine le badge ?

Vous devez voir : la grille dessinée sur la page ; avec `1fr 2fr`, une colonne deux fois plus large que l'autre.

Perdu ? Copiez `rattrapage/grid/etape-4/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Enregistrer et envoyer (2 min)
Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-06` :

```
git status
git add pas-a-pas-grid
git commit -m "Ajoute la grille de projets"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

Exercices 1 et 2 — La galerie et la page (30 min)

1. Exercice 1 : ensemble. Exercice 2 : seuls.
2. Dossier `grid-exercices`, fichier `style.css`.
3. **Réussi si** : les tests de ces deux exercices passent.

Niveau : guidé.

```
node --test grid-exercices.test.js
```

> **Aides**
>
> - **Le cours** : Mardi, « Apport — Grid ».
> - **L'IA** : CS50.ai (palier 1) : demandez une explication, pas le code.
> - **MDN** : Grilles ; `grid-template-columns`.

Enregistrer et envoyer (2 min)
Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-06` :

```
git status
git add grid-exercices
git commit -m "Ajoute les exercices Grid"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

À retenir — ce matin

> **À retenir**
>
> - Grid : deux dimensions, sur le parent.
> - `1fr` : une part de l'espace disponible.
> - Flexbox pour une rangée, Grid pour une grille.

Si le temps le permet

- Exercice 3 de `grid-exercices` (semi-guidé), les tarifs : **Réussi si** les trois tests passent.
- Les niveaux suivants de Grid Garden, jusqu'au 28.
- Le défi 6 du dossier `defis-css` (semaine 5), le damier : la grille que l'on n'avait pas encore mercredi.

ℹ Aides

- **Le cours** : Mardi, « Apport — Grid ».
- **L'IA** : CS50.ai (palier 1) : demandez une explication, pas le code.
- **MDN** : Grilles ; `grid-template-columns`.

Si le temps le permet — Grid Garden

1. Ouvrez Grid Garden, version du cours : le jeu s'affiche en français.
2. Arrosez les carottes en plaçant le terrain en CSS Grid.
3. **Réussi si** : le niveau 12 au moins ; capture du dernier niveau réussi.

Niveau : guidé.

Grid Garden existe en français et compte 28 niveaux (Thomas Park, s. d.). Cette copie est hébergée avec le cours, sans mesure d'audience ; votre progression reste dans votre navigateur. Code : MIT, © Thomas Park. Images : Creative Commons BY 3.0, © Thomas Park.

ℹ Aides

- **Le cours** : Mardi, « Apport — Grid ».
- **L'IA** : CS50.ai (palier 1) : demandez une explication, pas le code.
- **MDN** : Grilles.

Débrief

- Flexbox ou Grid pour une galerie de 9 photos ?

Après-midi — Le responsive : mobile first

Apport — Mobile first (15 min)

- **Pourquoi** : la majorité des visiteurs arrivent sur un téléphone.
- **Mobile first** : les styles de base pour le téléphone, puis on enrichit.

Pourquoi. Commencer par le petit écran oblige à décider ce qui compte ; le grand écran ajoute de la place, pas de nouvelles idées.

Définition. En mobile first, les styles de base visent le téléphone (une seule colonne) ; une media query en `min-width` ajoute les styles des grands écrans (MDN Web Docs, s. d.-c). La balise `meta name="viewport"` de lundi est indispensable pour que le téléphone utilise sa vraie largeur (MDN Web Docs, s. d.-d).

Exemple.

```
.projets { display: grid; grid-template-columns: 1fr; }

@media (min-width: 48rem) {
  .projets { grid-template-columns: repeat(3, 1fr); }
}
```

Résultat : une page qui utilise ce code (un menu en Flexbox, trois projets en Grid, la media query `min-width: 48rem`), à trois largeurs.



Figure 8. – 375 px, un téléphone : une colonne



Figure 9. – 768 px, une tablette : la media query s'applique, trois colonnes



Figure 10. – 1280 px, un ordinateur : trois colonnes, contenu centré

Pas à pas.

1. Écrivez la règle de base : une colonne.
2. F12, mode responsive : réglez la largeur à 375 px ; une colonne.
3. Ajoutez la media query ; élargissez à plus de 768 px (48 rem) : trois colonnes.

Mobile first — attention et vérification

> ⚠ Attention
>
> Une media query en `max-width` (héritée d'un tutoriel) inverse la logique : le téléphone reçoit des corrections au lieu des styles de base. Ici, toujours `min-width`.

- **Vérifiez** : à 375 px, combien de colonnes dans l'exemple ?

Pas à pas ensemble — La page en deux largeurs (55 min)

- Je code, vous tapez en même temps : une étape à la fois.
- Perdu ? Dossier `rattrapage/responsive/`.
- Dossier : `pas-a-pas-responsive`.
- F12, mode responsive, ouvert du début à la fin.

Niveau : guidé. Ouvrez `pas-a-pas-responsive/index.html` avec Live Preview, puis le mode responsive des outils (F12).

Étape 1. Écrivez les styles de base : une colonne. Réglez F12 à 375 px, en mode responsive.

Avant d'enregistrer : à 375 px, combien de colonnes ?

```
/* Mobile first : la grille, pour le téléphone */
.projets {
  /* Grid : les enfants se placent en grille */
  display: grid;
  /* Mobile first : une seule colonne */
  grid-template-columns: 1fr;
  /* Grid : un espace entre les cartes */
  gap: var(--espace-3);
}
```

Vous devez voir : une colonne à 375 px ; et encore une seule à 1280 px : c'est le départ.

Perdu ? Copiez `rattrapage/responsive/etape-1/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Étape 2. Ajoutez la media query, à la fin du fichier. Élargissez, puis réduisez.

Avant d'enregistrer : à 1280 px, combien de colonnes ? Et à 375 px ?

```
/* Media query : à partir de 48rem (768 px) */
@media (min-width: 48rem) {
  /* Grand écran : la grille change */
  .projets {
    /* Grid : trois colonnes égales */
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Vous devez voir : trois colonnes à 1280 px, une seule à 375 px. Le changement se fait à 768 px.

Perdu ? Copiez `rattrapage/responsive/etape-2/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Étape 3. Écrivez les styles de base de l'en-tête, **au-dessus** de la media query : le logo au-dessus du menu.

```
/* Mobile first : l'en-tête, pour le téléphone */
.entete {
  /* Flexbox : l'en-tête est un parent flex */
  display: flex;
  /* Flexbox : logo au-dessus du menu */
  flex-direction: column;
  /* Flexbox : un petit espace entre les deux */
  gap: var(--espace-2);
}

/* Flexbox : le menu est un parent flex */
.menu {
  /* Flexbox : les liens se placent en ligne */
  display: flex;
  /* Flexbox : un espace entre les liens */
  gap: var(--espace-3);
}
```

Vous devez voir : à 375 px, le logo au-dessus du menu.

Perdu ? Copiez `rattrapage/responsive/etape-3/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Étape 4. Dans la media query existante, ajoutez la règle des grands écrans.

Avant d'enregistrer : où va le logo à 1280 px ?

```
/* Media query : à partir de 48rem (768 px) */
@media (min-width: 48rem) {
  /* Grand écran : l'en-tête change */
  .entete {
    /* Flexbox : logo et menu côte à côte */
    flex-direction: row;
    /* Axe principal : un enfant à chaque bout */
    justify-content: space-between;
    /* Axe secondaire : centrés en hauteur */
    align-items: center;
  }

  /* Grand écran : la grille change */
  .projets {
    /* Grid : trois colonnes égales */
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Vous devez voir : à 1280 px, le logo à gauche et le menu à droite ; à 375 px, rien n'a changé.

Perdu ? Copiez `rattrapage/responsive/etape-4/style.css` à la place de votre `style.css`, puis repartez avec le groupe.

Enregistrer et envoyer (2 min)
Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-06` :

```
git status
git add pas-a-pas-responsive
git commit -m "Ajoute la page en mobile first"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

Exercices — La galerie et le menu (35 min)

1. Exercice 1 : ensemble. Exercice 2 : seuls (semi-guidé).
2. Dossier `responsive-exercices`, fichier `style.css`.
3. **Réussi si** : `node --test responsive-exercices.test.js` passe.

Niveau : semi-guidé.

> ℹ Aides
>
> - **Le cours** : Mardi, « Apport — Mobile first ».
> - **L'IA** : CS50.ai (palier 1) : demandez une explication, pas le code.
> - **MDN** : Responsive design ; Media queries.

« Layout Lab » : votre page personnelle (25 min)

1. Copiez votre page dans `semaine-06`.
2. En-tête et menu : Flexbox ; projets ou compétences : Grid.
3. Une media query en `min-width` pour les grands écrans.
4. **Réussi si** : `node --test layout.test.js` passe (fin jeudi).

Niveau : semi-guidé.

Depuis `~/dwwm/dwwm-tp/semaine-06`, copiez votre page de la semaine dernière par-dessus le fichier de départ du kit :

```
cp -r ../semaine-05/page-perso/. page-perso/
```

Vous devez voir : dans VS Code, votre `page-perso/index.html`, et non le fichier de départ. Ce travail continue jeudi, dans le parcours d'autonomie.

> **Aides**
>
> - **Le cours** : Mardi, « Apport — Mobile first » ; Lundi, « Apport — Flexbox » ; Mardi, « Apport — Grid ».
> - **L'IA** : CS50.ai (palier 1) : demandez une explication, pas le code.
> - **MDN** : Responsive design ; Media queries.

Enregistrer et envoyer (2 min)
Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-06` :

```
git status
git add responsive-exercices page-perso
git commit -m "Commence la mise en page en mobile first"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

À retenir — cet après-midi

> **À retenir**
>
> - D'abord le téléphone : une colonne.
> - Ensuite `@media (min-width: 48rem)`.
> - Toujours vérifier à 375 et à 1280 px.

Si le temps le permet

Avancez le Layout Lab.

Défi

Le terrain d'essai Flexbox et Grid : changez chaque réglage, prédisez le résultat **avant** de regarder, puis recopiez dans votre page le code affiché qui vous sert (15 min).

Aides

- **Le cours** : Mardi, « Apport — Mobile first ».
- **L'IA** : CS50.ai (palier 1) : demandez une explication, pas le code.
- **MDN** : Boîtes flexibles ; Grilles.

Sources

MDN Web Docs. s. d.-a. « Grilles - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/CSS_layout/Grids.

MDN Web Docs. s. d.-b. « Les concepts de base des grilles - CSS ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/CSS/Guides/Grid_layout/Basic_concepts.

MDN Web Docs. s. d.-c. « Media queries - CSS ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/CSS/Guides/Media_queries/Using.

MDN Web Docs. s. d.-d. « Responsive design - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/CSS_layout/Responsive_Design.

Thomas Park. s. d. « Grid Garden - A game for learning CSS grid layout ». Consulté le 26 septembre 2026. https://cssgridgarden.com/#fr.